

Stack Overflow is a community of 4.7 million programmers, just like you, helping each other.

Join them; it only takes a minute:

Sign up

Join the Stack Overflow community to:

Ask programming questions

Answer and help your peers

Get recognized for your expertise

How could I order by sum, within a DataFrame in PySpark?



Analogously to:

```
order_items.groupBy("order_item_order_id").count().orderBy(desc("count")).show()
```

I have tried:

```
order_items.groupBy("order_item_order_id").sum("order_item_subtotal").orderBy(desc("sum")).s
```



but this gives an error:

```
Py4JJavaError: An error occurred while calling o501.sort : org.apache.spark.sql.AnalysisException: cannot resolve 'sum' given input columns order_item_order_id, SUM(order_item_subtotal#429);
```

I have also tried:

```
order_items.groupBy("order_item_order_id").sum("order_item_subtotal").orderBy(desc("SUM(orde
```



but I get the same error:

```
Py4JJavaError: An error occurred while calling o512.sort : org.apache.spark.sql.AnalysisException: cannot resolve 'SUM(order_item_subtotal)' given input columns order_item_order_id, SUM(order_item_subtotal#429);
```

I get the right result when executing:

```
order_items.groupBy("order_item_order_id").sum("order_item_subtotal").orderBy(desc("SUM(orde
```



but this was done *a posteriori*, after having seen the number that Spark appends to the sum column name, i.e. **#429**.

Is there a way to get the same result but *a priori*, without knowing which number will be appended?

python pyspark spark-dataframe

asked Dec 21 '15 at 11:35

Albert Villanova del Moral

6 1

1 Answer

You should use aliases for your columns:

```
import pyspark.sql.functions as func

order_items.groupBy("order_item_order_id")\
    .agg(func.sum("order_item_subtotal")\
        .alias("sum_column_name"))\
    .orderBy("sum_column_name")
```

answered Dec 21 '15 at 12:07

architectonic

352 1 10

Thank you. To get also the column "order_item_order_id" (analogously to the output in the case with the count function), I have added this as an argument to the agg function:

```
order_items.groupBy("order_item_order_id")\
    .agg(order_items["order_item_order_id"],\
    func.sum("order_item_subtotal")\
    .alias("sum_column_name"))\
    .orderBy("sum_column_name")
```

Nevertheless, I wonder if there is a simpler way to get the same result, as in the example for count. – [Albert Villanova del Moral](#) Dec 21 '15 at 16:16

You don't need to complicate things, just use the code provided:

```
order_items.groupBy("order_item_order_id").agg(func.sum("order_item_subtotal").\
alias("sum_column_name")).orderBy("sum_column_name")
```

I have tested it and it works. – [architectonic](#) Dec 21 '15 at 17:25

Your code gives as output only the column "sum_column_name" and I also need the column "order_item_order_id". That is the reason I had to add this column inside the agg:

```
order_items.groupBy("order_item_order_id").agg(order_items["order_item_order_id"],\
func.sum("order_item_subtotal").alias("sum_column_name")).orderBy("sum_column_name")
```

– [Albert Villanova del Moral](#) Dec 22 '15 at 12:15

Oh I see, you're right about that Please consider using `cube` if you need multiple levels of aggregation with different aggregation functions. For example, you could write: `order_items.cube('order_item_order_id', 'some_other_column').agg({'order_item_subtotal': 'sum', '*': 'count'})` – [architectonic](#) Dec 22 '15 at 13:41